

Distributed Real Time Control Systems - Stage 2

Beatriz Coelho (107113), Luís Nobrega (106716), Maria Abrantes (106339)

I. REAL TIME ARCHITECTURE

A. Dual core architecture

To avoid overwhelming the default core 0 with both communication and control tasks, both RP2040 cores were used. Core 0 was responsible for communication and I/O (slower tasks) while Core 1 was reserved for PID and distributed algorithm calculations (Tab. I). This allows for CAN processing without affecting calculations.

B. Shared state and synchronization

Both cores share RAM, so we used global volatile shared variables. For non atomic operations, local copies were created and used. This way, we avoid core 1 reading values that are being updated by core 0 and vice-versa.

To ensure integrity across both cores without the overhead of an RTOS, we implemented two hardware spinlocks: *CAN_LOCK()*, which serializes access to the MCP2515 controller via SPI (and prevents CAN collisions), and *PID_LOCK()*, protecting PID calculations from concurrent access. This bare-metal approach proved efficient and an alternative to task scheduling. Small delays were introduced between consecutive transmissions to prevent MCP2515 buffer saturation. This proved to be critical.

C. MCP2515 buffer handling and CAN error Handling

Using only RXB0 lead to communication failures. To further improve CAN capacity and avoid losing messages, we enabled use of both RXB0 and RXB1 as well as of TX0, TX1 and TX2.

An error-checking mechanism was implemented by exposing the MCP2515's three diagnostic registers — the Transmit Error Counter (TEC, 0x1C), Receive Error Counter (REC, 0x1D), and Error Flag Register (EFLG, 0x2D). The function *can_check_errors()* reads these registers every 5 seconds from core 1 and prints a labelled report if TEC, REC, or any error flag is non-zero. It also detects Bus-Off, TX/RX Error-Passive, TX/RX Error Warning, and RX buffer overflow conditions. In the overflow case, the overflow flags are automatically cleared so the chip can resume accepting messages.

D. CAN Optimisation

CAN messages take time to travel between controllers ($\approx 100\mu s$) depending on the data sent. To reduce delays we use binary encoding via *memcpy* rather than sending values as strings. This way, floats take 4 bytes instead of 7 (ASCII). Furthermore, we offloaded node addressing to the 11-bit CAN Identifier (0x100 + Destination ID). While current implementation uses a 'Type' byte within the data field, further efficiency could be gained by bit-masking the CAN ID, which would allow for sending 2 floats (8 bytes).

E. Concurrent programming and FreeRTOS alternative

A function was implemented to monitor real-time performance, tracking average and worst-case execution times for the PID compute, PID period jitter, and CAN handle. Sending too many reference requests delayed the system a few ms.

Give that with delay usage is not ideal, we tried to switch to a task approach with FreeRTOS. Printing were moved to a low priority *LOG_TASK*, then high priority *RX_TASK* and *TX_TASK* were added as message handles between txQueue and rxQueue while *CAN_PROCESS_TASK* would process the received messages. This way no messages would be lost. Ideally, both these would wake up based on queue/hardware ISR, however, the interrupt based on the INIT pin wasn't working as expected. The remaining implementation separated the existing code into *SENSOR_TASK*, *PID_TASK*, *CONVERGENCE_TASK* and *CALIB_TASK* (Tab. VII). We also used a series of *semaphores* to avoid race conditions (Tab. VIII) This approached ultimately failed, due to insufficient time.

II. SYSTEM WAKE-UP AND CALIBRATION

A. Node Recognition

A wake up procedure was developed to detect all other nodes, with the algorithm prepared to deal with arbitrary N nodes. At start-up, each node broadcasts its unique hardware identifier through the CAN bus and stores the IDs received from other active nodes. The nodes either wait an interval of 10s, or start early if every node broadcasts that it has found the maximum node number (predefined by the user). The identifiers are then ordered and logical node IDs are assigned accordingly. Once the number of active nodes is known, the required memory is allocated according to N for the node list, calibration terms, shared duty-cycle vectors, and the coupling gain matrix.

B. Calibration Procedure

The introduction of multiple LEDs and sensors naturally introduces cross-coupling effects, which need to be taken into account for the correct modelling of the plant. The total illuminance y_i measured by a given node is thus given by:

$$y_i = \sum_{j=1}^N G_{ji} u_j + d_i, \quad (1)$$

where i is the sensor index, j is the LED index, G_{ji} represents the cross coupling gain from LED j to sensor i ¹, $u \in [0, 1]$ is the duty cycle applied to LED j , and d corresponds to the background disturbance measured by sensor i .

The calibration procedure used a *non-blocking State Machine*, where LEDs sequentially turn on and off while the sensors record the resulting illuminance. The node with the lowest ID is selected to set $u = 1$ (ON). After waiting 4s for the LDR sensors

¹The presented notation is the one applied in the project, and may differ from the standard.

to stabilize, it measures its own illuminance, and broadcasts a command for the remaining nodes to record theirs. Once every node saved the measured value, the selected node passes the token to the next lowest ID node. The sequence repeats until the highest ID node has taken its turn.

When this procedure finishes, every node has saved the effect of every LED on itself, which corresponds to a single column of the matrix G . It is important to note that because the system is fully distributed: each node only knows what it has measured, rather than the entire matrix G .

Finally, a flag is activated to exit the calibration phase and initiate the main control loop. Table II provides a description of the states used in the state machine. Tables III, IV, V and VI provide additional information on information received, commands exchanged as well as a small demo with 3 MCUs.

III. DISTRIBUTED CONTROLLER

A. FFAD: a Competitive Approach

To address the distributed control problem, the Feedforward from Accessible Disturbance (FFAD) algorithm was selected. It is important to note that the FFAD is a competitive algorithm, where each node independently attempts to drive its own tracking error ($e = r - y$) to zero. Thus, each node treats the cross coupling effects from the remaining LEDs as accessible disturbances. Because there is no a common goal (no global cost function), the nodes act selfishly and do not cooperate.

Consequently, FFAD cannot ensure a minimum illuminance across all desks. Consider a scenario in which one of the nodes is given a physical unreachable reference (very high), while the remaining nodes are asked for very low illuminances. Once working selfishly, the first node will saturate ($u = 1.0$), while the others will lower their outputs to meet their own minimal targets. This way, these nodes will not "help" the first one, by increasing their own duty cycles and produce more light. Consequently, the minimum illuminance requirement for the first node will fail to be met.

To solve this issue, cooperative algorithms such as Dual Decomposition² or Alternating Direction Method of Multipliers (ADMM) would have to be implemented, as they address this as a global optimization problem, with constraints.

Although being the simplest of the considered approaches, FFAD is easy to understand and to debug, while proving convergence. The algorithm was applied sequentially, rather than simultaneously, as the convergence is proved to be faster in the chosen case.

B. Implementation

The implemented logic consists of a two-phase approach: a sequential FFAD to compute the ideal duty cycles, followed by a local PID to reject environmental disturbances and modelling imperfections. Whenever a new reference illuminance (r_i) is set, all nodes suspend closed-loop control and enter the "convergence phase". In this phase, nodes take turns to calculate their ideal duty cycle (u_{ideal}), using the gains and background disturbances recorded during calibration. The ideal duty cycle for node i is given by:

$$u(ideal)_i = \frac{r_i - d_i - \sum_{j \neq i}^N G_{ji} u_j}{G_{ii}} \quad (2)$$

²We tried implementing this algorithm, but failed due to lack of time.

Equation 2 demonstrates that each node compensates for the current light emitted by the other LEDs in the network. After computing this, the selected node broadcasts its ideal duty cycle to the remaining nodes. The node with the following ID number is chosen to take the next turn (similar to the calibration) and compute its own $u(ideal)$.

This sequence repeats until convergence is reached. For this implementation, convergence is considered to be reached after a finite number of cycles of the described process. To determine this number, a numerical simulation was conducted in *Matlab*. A total number of 6 full cycles (where 1 full cycle is defined as a complete sequence of updates across all nodes) seemed applicable, yielding convergence in all the simulated experiences. Results can be seen in Fig. 3.

In the system, the command `d` was used to print the last convergence record of the duty-cycle values u . Figure 4 consists of three consecutive convergence phases, applied in the real system. As observed, the duty cycles stop varying within 6 cycles after the beginning of each convergence phase. Furthermore, the expected illuminance plot confirms the correct tracking of the reference (ideally).

When the total number of cycles is reached, a bumpless transfer mechanism is used to incorporate the FFAD result with the local PID controllers. By defining the total control effort as $u_{total} = u_{ideal} + u_{PID}$, the PID is "relieved" from generating the full baseline control signal (as done previously). Hence the PID is responsible for correcting ambient disturbances and modelling errors.

Figures 5 and 6 illustrate the system's behaviour across three distinct stages: initial closed-loop PID control, FFAD convergence, and final PID control.

In Figure 5, node 2's initial target is 10 Lux. However, cross-coupling effects from the other nodes forces node 2's measured illuminance above its target (being turned OFF isn't sufficient). When node 2's reference is increased to 25 Lux, the systems enter the convergence phase and new duty cycles are computed. Those are then applied to each LED and, once closed-loop control resumes, the PIDs smoothly adjust them to correctly match the reference.

Figure 6 demonstrates the algorithm's competitive nature, as have been discussed previously. Initially, all nodes successfully follow low reference values. Then, a physically unreachable reference is applied to node 3. During convergence phase, the optimal solution is found almost immediately, as it is simply for node 3 to saturate at maximum power ($u = 1.0$), while nodes 1 and 2 turn off to compensate the strong cross coupling light. Ultimately, node 3 fails to reach its target, while nodes 1 and 2 remain above their target. This example confirms the competitiveness of the FFAD algorithm, where every node attempts to reach its own target.

IV. CONCLUSION

The implemented system combined CAN communication, dual-core execution, dynamic node recognition, and non-blocking calibration for a variable number of luminaires. Using an FFAD-based distributed strategy together with local PID control, the setup was able to demonstrate wake-up, calibration, convergence, and closed-loop operation despite communication delays.

APPENDIX A

APPENDIX B

Table I: Dual-Core Responsibility Summary

Core	Role	Responsibilities
Core 0	Communication & Management	• Node identification • CAN communication handling • Message transmission and reception • System calibration procedures
Core 1	Control & Computation	• PID calculations • Concurrent programming tasks • Convergence computation of control input (u)

Table II: Calibration States: Actions and Transitions

State	Duration	Boss Actions	Follower Actions
0	Wait 10 sec	Broadcast 'w' (wakeup) every 1 sec Collect node UIDs Sort UIDs, allocate arrays Assign node_id values Set boss_node = 1	Receive wakeup 'w' frames Add new UIDs to list (No action, just wait)
1	Immediate	Send calibration start 'c' Turn LED OFF: duty_cycle = 0 Disable feedback Broadcast state change 'S'=2	Receive 'c' frame Turn LED OFF Wait for 'S'=2
2	2 seconds	Wait 2 sec (LED stabilizes) Turn LED ON: duty_cycle = 1.0 Broadcast state change 'S'=3	Receive 'S'=2 (Stay in state 2, wait) Prepare to measure
3	2 seconds	Wait 2 sec (output stabilizes) Record lux: Gs[<i>boss</i>][<i>boss</i>] Send 's' (store lux request) Broadcast 'S'=5	Receive 'S'=3 Measure light from boss Receive 's' frame Record: Gs[<i>boss</i>][<i>me</i>]
5	300 ms wait	Wait 300 ms (propagation guard) LED OFF: duty_cycle = 0 Find next node to be boss If more nodes: go to state 1 If last node: go to state 6	Receive 'S'=5 (No action) Wait for next 'S'=1 or 'S'=6
6	2 seconds	Wait 2 sec (ambient measurement) Record ambient: d_terms[<i>boss</i>] Send calibration end 'f' Enable feedback Broadcast 'S'=7	Receive 'S'=6 Measure ambient light Receive 'f' frame Set calib_st = 7
7	Immediate	Calculate true gains: Gs[<i>row</i>][<i>col</i>] -= d_terms[<i>col</i>] Clamp negatives to zero Print Gs matrix If boss: sv_my_turn = true Broadcast 'S'=8	Receive 'S'=7 Calculate true gains Print Gs matrix (Wait for state 8)
8	Final	Calibration complete Ready for control loop Boss gets first turn	Calibration complete Ready for control loop Wait for control turn

H]

Table IV: Calibration State Transitions and Conditions

From State	To State	Condition / Trigger
0	1	Wait 10 seconds → All nodes present, assign IDs
1	2	Boss sends 'c' and 'S'=2 Followers receive 'S'=2 via CAN
2	3	Wait 2 seconds, LED stabilizes, Boss sends 'S'=3 Followers receive 'S'=3 via CAN
3	5	Wait 2 seconds, measure lux, Boss sends 's' and 'S'=5 Followers receive 'S'=5 via CAN
5	1	Boss waits 300 ms, finds next node, sends 'S'=1 (new boss) Loop back: repeat for next node
5	6	Boss waits 300 ms, no more nodes found, sends 'S'=6
6	7	Wait 2 seconds, Boss measures ambient, sends 'f' and 'S'=7 Followers receive 'f' (set calib_st=7) Followers receive 'S'=7 via CAN
7	8	Calculate true gains, Boss sends 'S'=8 Followers receive 'S'=8 via CAN All nodes ready for control loop

Table III: CAN Messages Broadcast During Calibration

State	From	Message Type	To	Purpose
0	Boss	'w' (wakeup)	Broadcast	Collect node UIDs
1	Boss	'c' (calib start)	All	Tell followers to dim
	Boss	'S'=2 (state change)	All	Move to state 2
2	Boss	'S'=3 (state change)	All	Move to state 3
3	Boss	's' (store lux)	All	Followers snapshot lux
	Boss	'S'=5 (state change)	All	Move to state 5
5	Boss	'S'=1 (state change)	All	New boss, restart (if more nodes)
	Boss	'S'=6 (state change)	All	Last node, move to state 6
6	Boss	'f' (calib end)	All	Followers set calib_st=7
	Boss	'S'=7 (state change)	All	Move to state 7
7	Boss	'S'=8 (state change)	All	Calibration complete

Table V: Measurements and Data Recorded During Calibration

State	Measurement	Stored In
0	Collect node UIDs	uids[] array
1	(LED OFF, dark)	Preparation
2	(LED OFF → ON)	Transition
3	Boss illuminance (100% duty) Follower measurements from boss	Gs[boss][boss] Gs[boss][follower_i]
5	(LED OFF, cool down)	Preparation for next boss
6	Ambient light (LED OFF)	d_terms[boss]
7	Subtract ambient from all: Gs[i][j] -= d_terms[j]	True gain matrix Gs[] [] (corrected)
8	(Calibration complete)	Ready for control

Table VI: Example Calibration Sequence (3 Nodes): State Progression

Time	State	Boss	Action	Messages Sent
T+0 s	0	—	Broadcast wakeups, collect UIDs	'w' frames
T+10 s	1	Node 1	LED OFF, send start	'c', 'S'=2
T+12 s	2	Node 1	Wait 2 sec	—
T+14 s	3	Node 1	LED ON, wait 2 sec	'S'=3
T+16 s	3	Node 1	Measure all nodes	's', 'S'=5
T+17 s	5	Node 1	Cool down, find Node 2	'S'=1 (boss=2)
T+18 s	1	Node 2	LED OFF, send start	'c', 'S'=2
T+20 s	2	Node 2	Wait 2 sec	—
T+22 s	3	Node 2	LED ON, measure	's', 'S'=3, 's', 'S'=5
T+23 s	5	Node 2	Cool down, find Node 3	'S'=1 (boss=3)
T+24 s	1	Node 3	LED OFF, send start	'c', 'S'=2
T+26 s	2	Node 3	Wait 2 sec	—
T+28 s	3	Node 3	LED ON, measure	's', 'S'=3, 's', 'S'=5
T+29 s	5	Node 3	No more nodes, go to 6	'S'=6
T+31 s	6	Node 3	Measure ambient	—
T+33 s	6	Node 3	Send end signal	'f', 'S'=7
T+34 s	7	All	Calculate gains	—
T+35 s	8	All	Ready for control	'S'=8

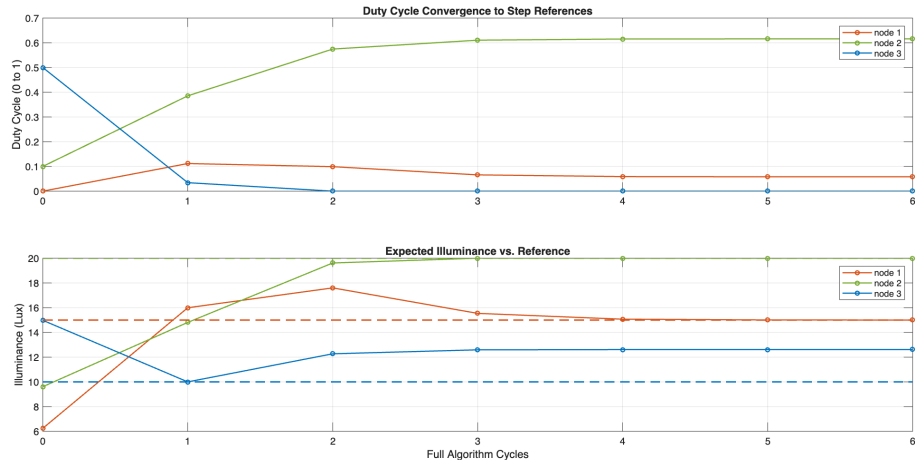
H]

Table VII: System Tasks, Cores, Priorities, and Functions

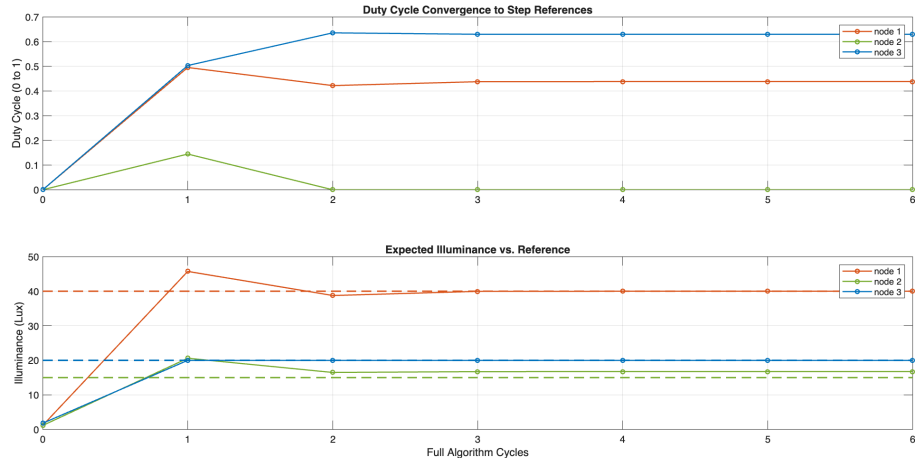
Task Name	Core	Priority	Stack (bytes)	Function
TASK_SENSOR	0	4	2048	Samples LDR sensor every 1 ms; updates <code>sv_y_lux</code>
Task_CAN_RX	0	3	2048	Polls MCP2515 every 1 ms; reads received CAN messages into <code>rxQueue</code>
Task_CAN_TX	0	2	2048	Dequeues messages from <code>txQueue</code> ; transmits to MCP2515 hardware
TASK_CAN_PROCESS	0	3	4096	Runs every 10 ms; drains <code>rxQueue</code> and dispatches messages by type
TASK_SERIAL_CMD	0	2	4096	Listens to serial input; parses commands; sends CAN messages
TASK_CALIB	0	2	4096	State machine for system calibration; measures gains and offsets
TASK_PID	1	4	4096	Runs every 10 ms; executes PID control loop; adjusts LED duty cycle
TASK_CONVERGENCE	1	3	4096	Implements distributed convergence algorithm via CAN
TASK_LOGGER	0	1	2048	Drains <code>logQueue</code> ; outputs debug messages to serial

Table VIII: Mutexes and Semaphores

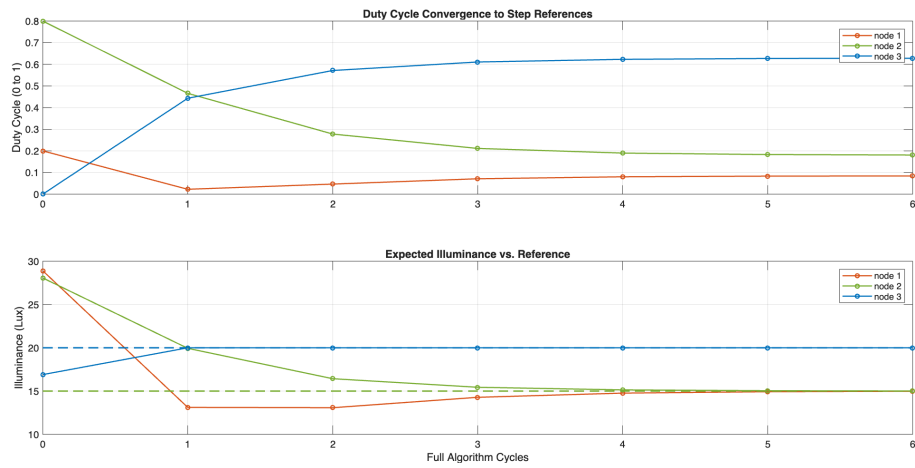
Primitive	Purpose
<code>stateMutex</code>	Protects scalar state variables: <code>sv_r</code> , <code>sv_duty_cycle</code> , <code>sv_feedback_on</code> , <code>sv_y_lux</code> , <code>sv_calib_st</code> , <code>sv_occupancy</code>
<code>pidMutex</code>	Protects <code>my_pid</code> object exclusively; guards <code>compute_control()</code> , <code>housekeep()</code> , <code>bumpless_transfer()</code>
<code>canRxSemaphore</code>	(Unused in current implementation; reserved for interrupt-driven RX)
<code>turnSemaphore</code>	Binary semaphore; signals convergence task to begin control turn



Simulation Example 1



Simulation Example 2



Simulation Example 3

Figure 3: Simulation Results: different sets of initial conditions and different sets of reference targets were tested. Every simulation converged in less than 6 full cycles. Note that 1 full cycle is defined as a complete sequence of updates across all nodes

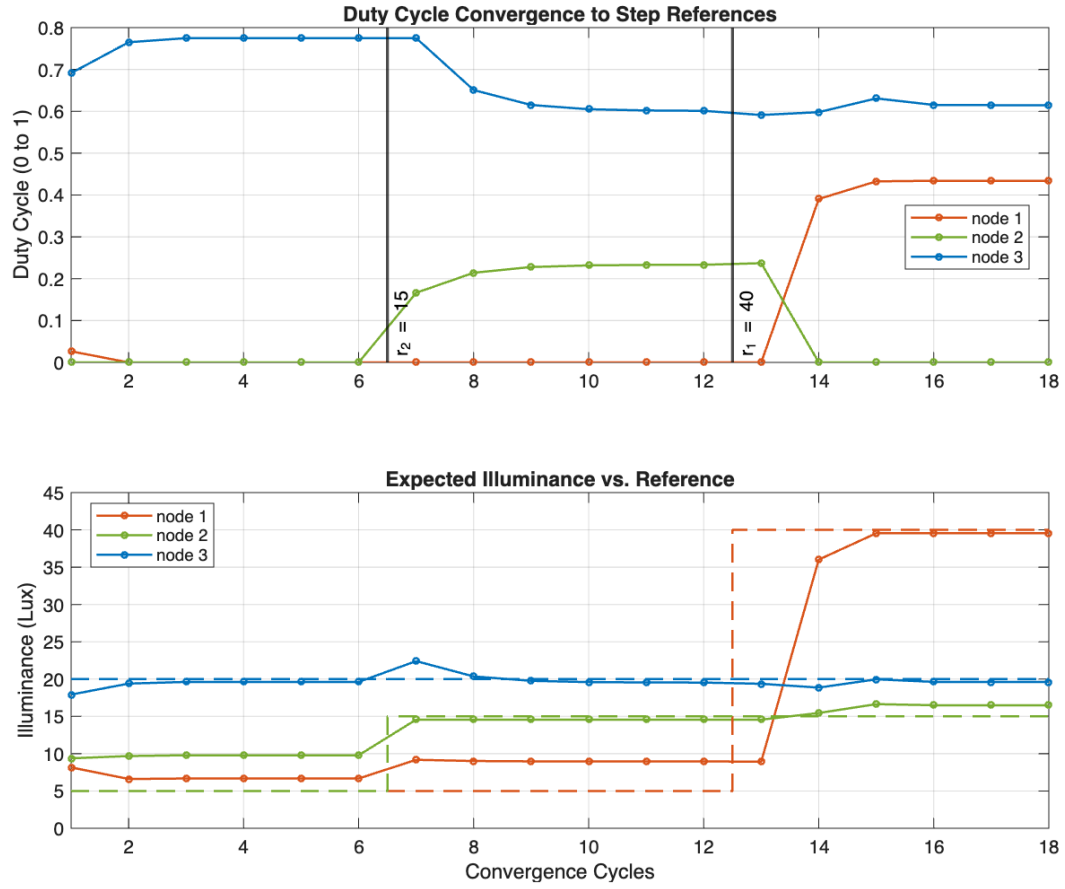
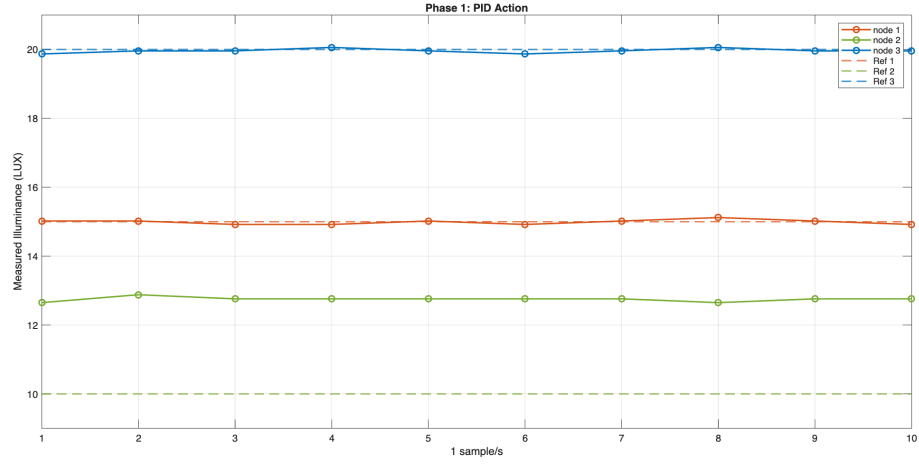
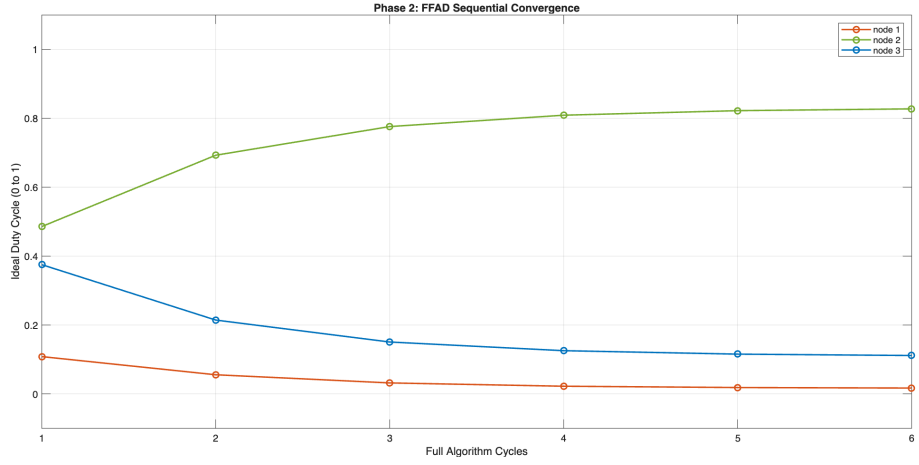


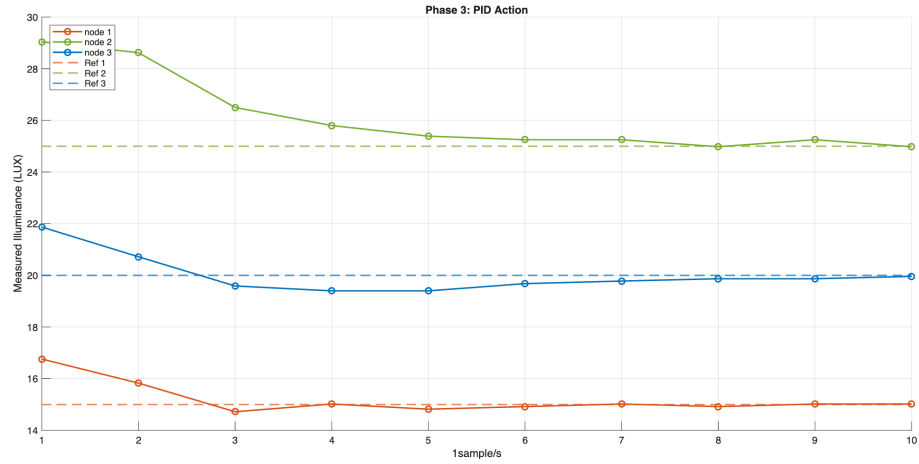
Figure 4: Convergence of the duty cycle in the real system. The vertical lines indicate the beginning of a new convergence phase. Initially, all references were 5 LUX (not shown in the figure). The sequence of reference targets applied is: $r_3 = 20$ LUX followed by $r_2 = 15$ LUX and finally $r_1 = 40$ LUX



(a) PID Action: the feedback controllers try to compensate for deviations to the target. Note that the PID action succeeded a convergence phase

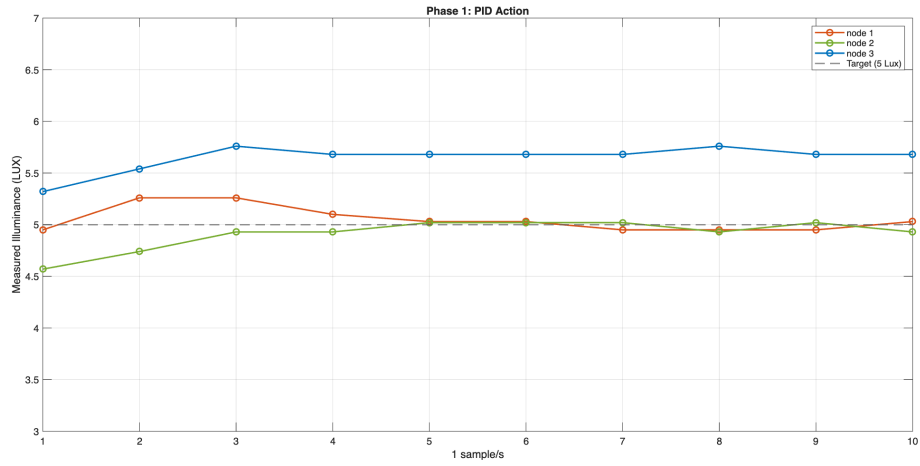


(b) A reference of 25 LUX was sent to node 2 (the other nodes' target remain $r_1 = 15$ LUX, $r_3 = 20$ LUX). The nodes enter the Convergence Phase.

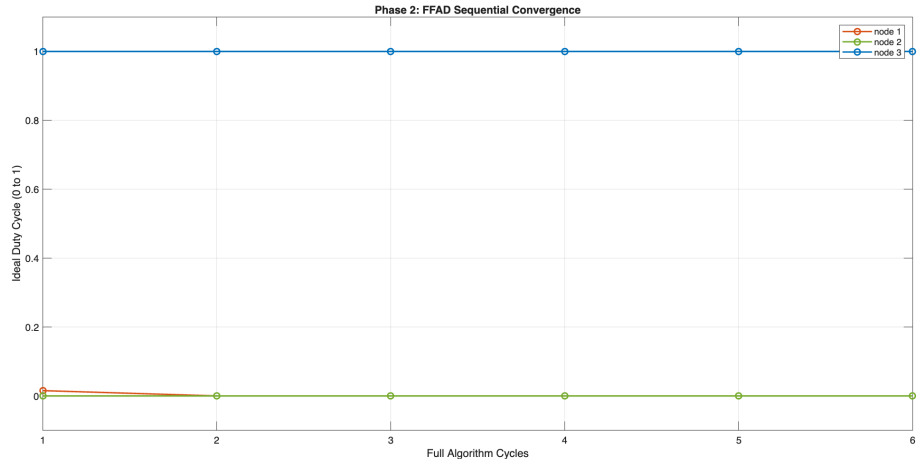


(c) PID Action: The final "agreed" values of u_{ideal} are applied to the LEDs and the PID controller makes the adjustments to ensure the correct tracking of the reference.

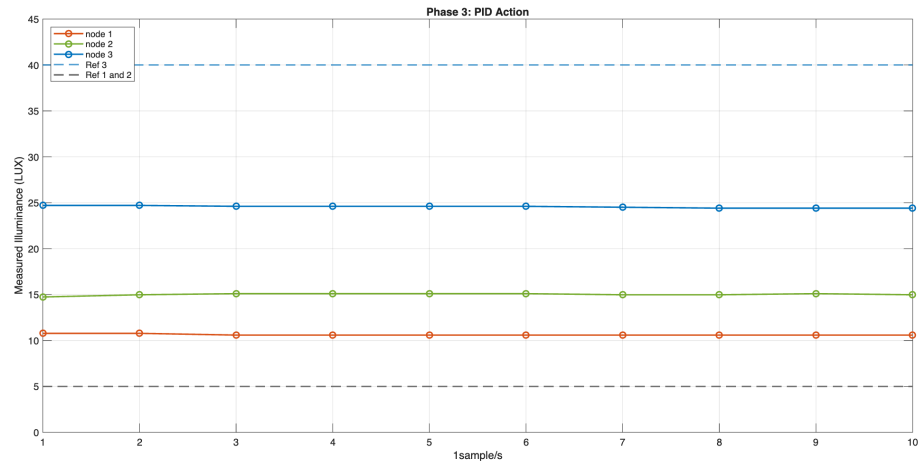
Figure 5: Sequence of a PID Action followed by a Convergence Phase, and finally PID Action again - Example 1



(a) PID Action: the feedback controllers try to compensate for deviations to the target. Note that the PID action succeeded a convergence phase



(b) A reference of 40 LUX was sent to node 3 (the other nodes' target remains at 5 LUX). The nodes enter the Convergence Phase.



(c) PID Action: The final "agreed" values of u_{ideal} are applied to the LEDs and the PID controller makes small adjustments.

Figure 6: Sequence of a PID Action followed by a Convergence Phase, and finally PID Action again. - Example 2

Table IX: Work division

Section	Responsible people
Dual-core architecture	Fernando
Concurrent programming	Fernando
N node generalization	Beatriz
CAN bus	Beatriz ; Fernando
Optimization	Fernando ; Maria
Calibration	Fernando ; Maria
Distributed controller	Maria
Data analysis	Maria
Task implementation + Dual Decomposition	Fernando

APPENDIX C

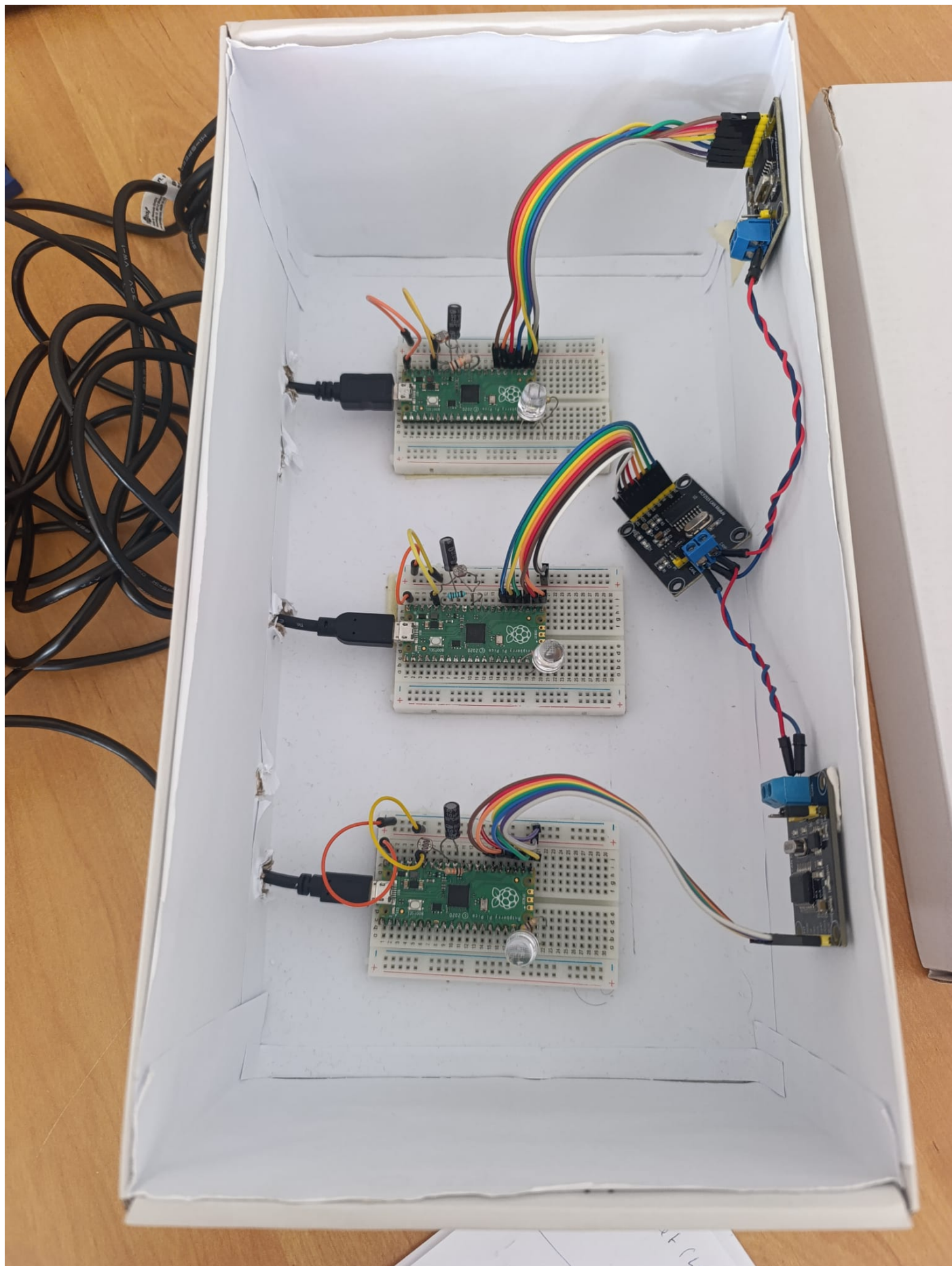


Figure 1: Luminaire setup seen from above. Extra holes were made on the side of the box to allow for cable passage and give room for creating disturbances with a flashlight.

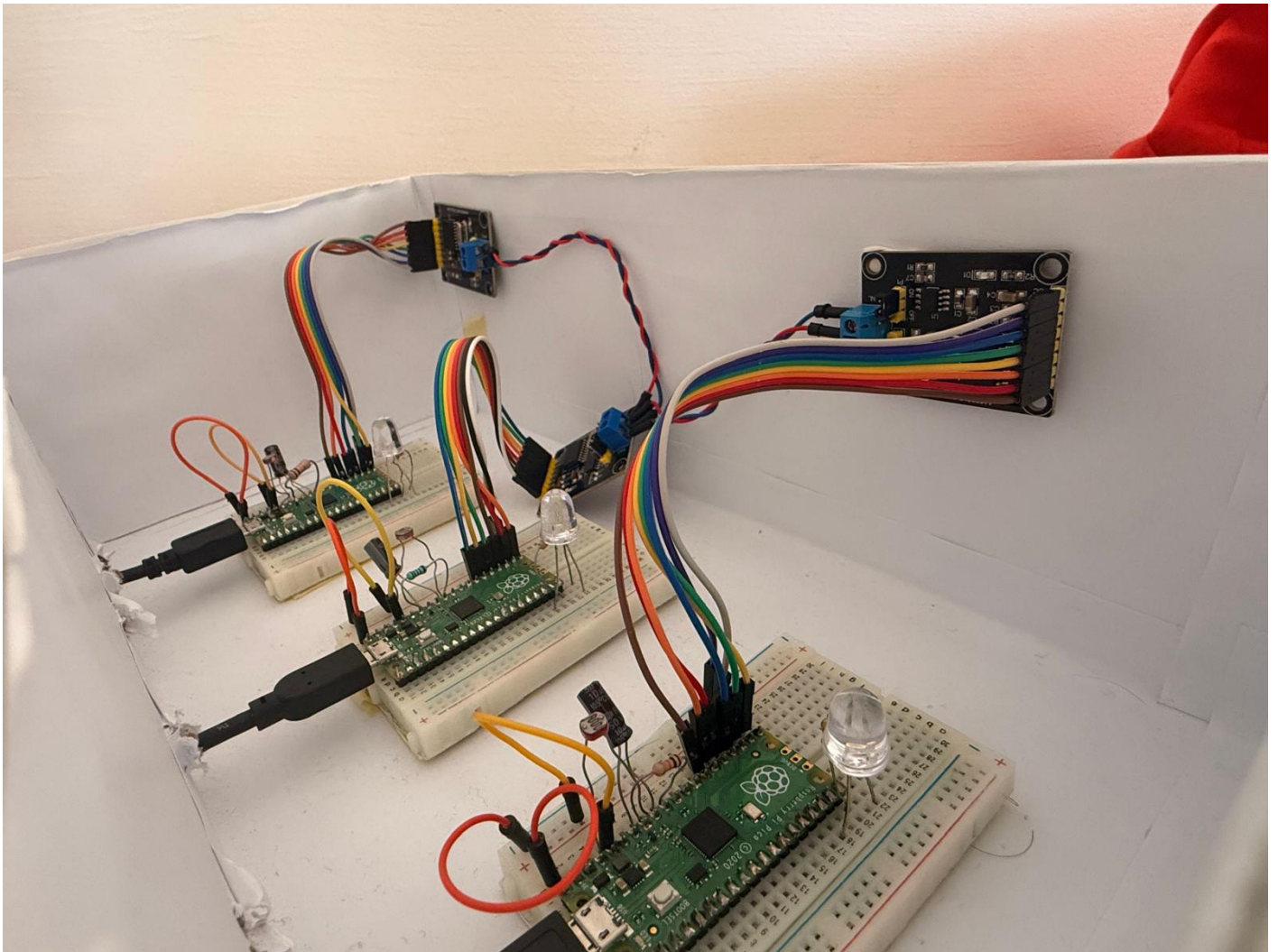


Figure 2: Luminaire setup, alternative perspective. CAN modules were glued with disposable materials to the side, hoping to minimize the amount of light blocked from one luminaire to the other.